



Name:- Jaykishan J Saharan.

Div:- D Roll no.: 26

Course/Program: BTech CSE SY

Subject:- DLM

DLM assignment 5.

Q.1) Architecture of 8086?

Solⁿ:- Microprocessor is a multipurpose, programmable device that accepts digital data as input processes it according to instructions stored in its memory, and provides results as output.

Q.2) Architecture of 8086 Microprocessor:

- As shown in the below figure, the 8086 CPU is divided into two independent functional parts-

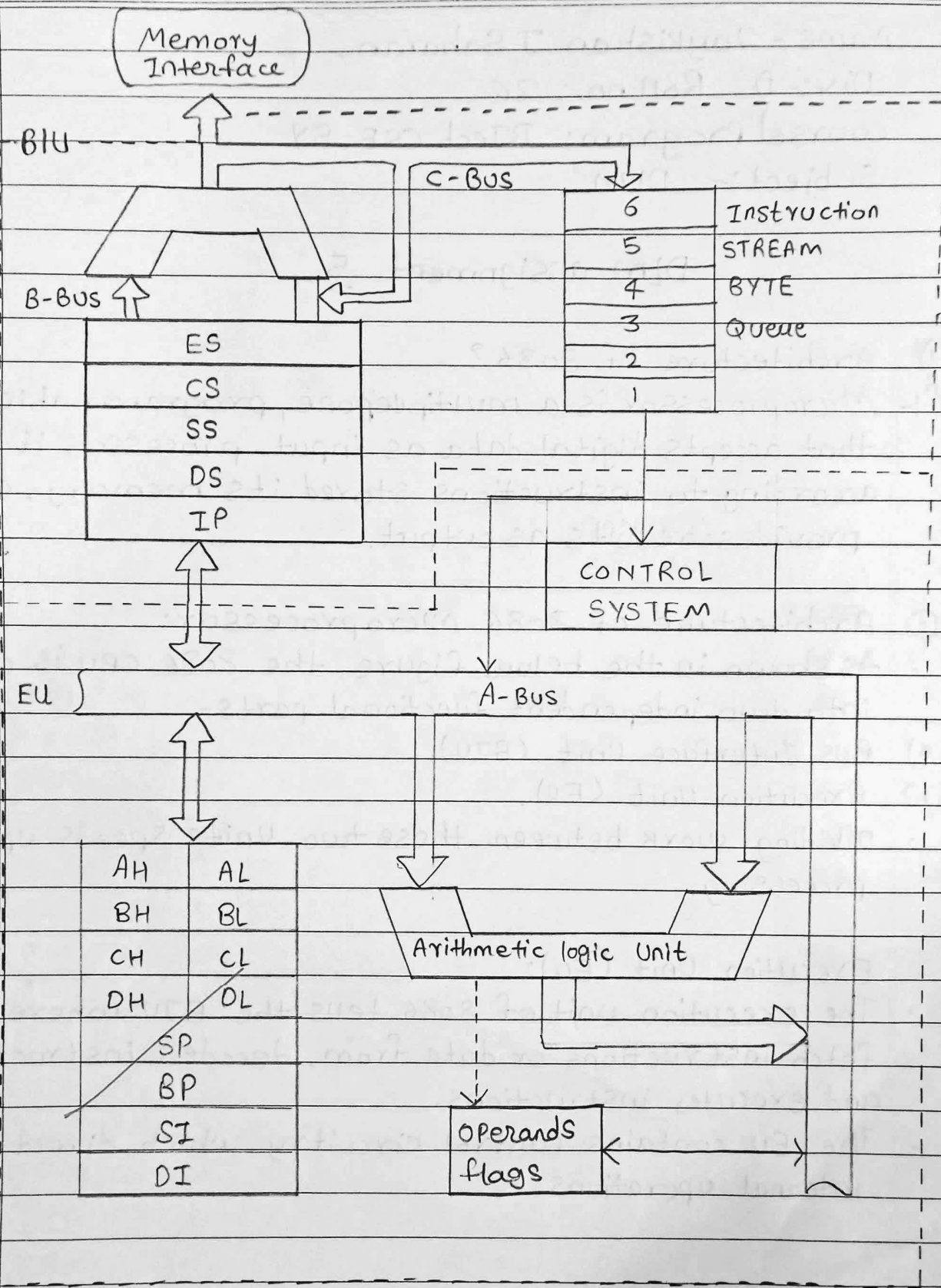
a) Bus Interface Unit (BIU).

b) Execution Unit (EU).

- Dividing work between these two units speeds up processing.

Execution Unit (EU):

- The execution unit of 8086 tells the BIU where to fetch instructions or data from, decodes instructions, and executes instructions.
- The EU contains control circuitry, which directs internal operations.





- A decoder in the EU translates instructions fetched from memory into a series of actions, which the EU carries out.
- The EU has a 16-bit arithmetic logic unit (ALU) which can add, subtract, AND, OR, XOR, increment, decrement, complement or shift binary numbers.
- The main function of EU are:-
 - a) Decoding of Instructions.
 - b) Execution of Instructions.

- Steps -

1. EU extracts instructions from top of queue in BIU
2. Decode the instructions.
3. Generates operands if necessary.
4. Passes operands to BIU & requests it to perform read and write bus cycles to memory or I/O
5. Perform the operation specified by the instruction on operands.

- The EU contains

1. ALU
2. General purpose registers.
3. Index registers.
4. Pointers
5. Flags register.

Bus Interface unit (BIU):

- The BIU Sends out addresses, fetches instructions from memory, reads data from ports and memory, and writes data to ports and memory.



- In simple words, the BIU handles all transfers of data and addresses on the buses for the execution unit.
- The BIU contains -
 1. Segment Registers
 2. Instruction Pointer (IP)
 3. Instruction Queue.

8086 has Pipelining Architecture:-

- While the EU is decoding an instruction or executing an instruction, which does not require use of the buses, the BIU fetches up to six instruction bytes for the following instructions.
- The BIU stores these pre-fetched bytes in a first-in first-out register set called a queue.
- When the EU is ready for its next instruction from the queue in the BIU. This is much faster than sending out an address to the system memory and waiting for memory to send back the next instruction byte or bytes.
- Except in the case of JMP and CALL instructions, where the queue must be dumped and then reloaded starting from a new address, this pre-fetch and queue scheme greatly speeds up processing.
- Fetching the next instruction while the current instruction executes is called pipelining.
- 8086 has a powerful set of registers known as general purpose registers and special purpose registers.



- All of them are 16-bit registers.
- General purpose registers:
 - 1) These registers can be used as either 8-bit registers or 16-bit registers.
 - 2) They may be either used for holding data, variables and intermediate results temporarily or for other purpose like a counter or for storing offset address for some particular addressing modes etc.
- Special purpose registers:

These registers are used as segment registers, pointers, index registers or as offset storage registers for particular addressing modes, etc.
- Special purpose registers:

These registers are used as segment registers, pointers, index registers or as offset storage registers for particular addressing modes.
- The 8086 registers are classified into the following types:
 1. General Data Registers.
 2. Segment Registers
 3. Pointer and Index Register
 4. Flag registers.

Q.2) Explain any 10 instructions of 8086?

Solⁿ:- 8086 Instructions:

- 1) mov (move): copies the byte or word from a source to a destination.
- MOV AX, BX - copies the content of Register BX to register AX
 - MOV CL, 50H - loads the immediate value 50H into the CL register.



- 2) **ADD (Add):** Adds a source operand to a destination operand and stores the result in the destination.
 - **ADD AX, 100H** - Adds the value 100H to the AX register.
- 3) **SUB (Subtract):** Subtracts the source operand from the destination operand.
 - **SUB CX, BX** - Subtracts the content of register BX from register CX.
- 4) **JMP (Jump):** Unconditionally transfers program control to a new address.
 - **JMP [label]** - Jumps to the instruction at the specified [label].
- 5) **PUSH (push):** Pushes a word from a register or memory onto the stack.
 - **PUSH AX** - Saves the contents of the AX register onto the stack and decrements the stack pointer.
- 6) **POP (POP)** - Pops a word from the stack into a register or memory location.
 - **POP CX** - Retrieves the top word from the stack and loads it into the CX register, then increments the stack pointer.



7) **CMP (compare)**: compares two operands and sets the flags based on the result of the subtraction without changing the operands.

- **CMP AX, BX** - compare the value in AX with the value in BX.

8) **INC (Increment)**: Increments the operand by 1.

- **INC DX** - Increases the value of register DX by 1.

9) **DEC (Decrement)**: decrements the operand by 1.

- **DEC SI** - Decrement the value of the source index register SI by 1.

10) **INT (Interrupt)**: Triggers a software interrupt to a specified vector, used for calling operating system or BIOS services.

- **INT 21H** - calls the DOS service routines.

~~C. Kulkarni~~